

R2.09 • **Projet KanbanRT • React + Supabase + Vercel**

Ce document résume l'architecture du projet, les pages, le routeur et les conventions de code.

1 — Arborescence du projet

Voici la structure de fichiers à respecter dans votre dépôt GitHub :

```
mon-kanban/
├─ public/
├─ src/
│   ├─ lib/
│   │   └─ supabase.js
│   ├─ pages/
│   │   ├─ LoginPage.jsx
│   │   └─ DashboardPage.jsx
│   ├─ components/
│   │   ├─ TaskCard.jsx
│   │   ├─ TaskForm.jsx
│   │   ├─ TaskList.jsx
│   │   └─ UserTable.jsx
│   └─ App.jsx
└─ main.jsx
.env.local
.gitignore
package.json
README.md
```

	Fichier	Rôle
	supabase.js	Initialise le client Supabase avec les clés d'environnement. Importé partout.
	App.jsx	Configure le routeur React Router. Protège les routes. Point d'entrée de l'app.

 <code>LoginPage.jsx</code>	Page publique : formulaire login/register. Redirige vers /dashboard si connecté.
 <code>DashboardPage.jsx</code>	Page protégée : navigation par onglets, affiche les tâches et les utilisateurs.
 <code>TaskCard.jsx</code>	Composant réutilisable : affiche une carte de tâche avec ses badges.
 <code>TaskForm.jsx</code>	Formulaire de création d'une tâche. Envoie un INSERT vers Supabase.
 <code>TaskList.jsx</code>	Charge et affiche les tâches. Contient TaskForm + liste de TaskCard.
 <code>.env.local</code>	Clés Supabase secrètes. Ne jamais commiter. Toujours dans .gitignore.

2 — Le client Supabase

Le fichier `src/lib/supabase.js` est le seul endroit où le client est créé. Tous les autres fichiers l'importent depuis là.

```
// src/lib/supabase.js

import { createClient } from '@supabase/supabase-js';

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseKey = import.meta.env.VITE_SUPABASE_ANON_KEY;

export const supabase = createClient(supabaseUrl, supabaseKey);
```



`import.meta.env`

Avec Vite, les variables d'environnement sont lues via `import.meta.env` (et non `process.env` comme avec Node.js). Elles doivent commencer par `VITE_` pour être exposées côté client.

Utilisation dans n'importe quel composant :

```
import { supabase } from '../lib/supabase';

// Lire des données
const { data, error } = await supabase.from('tasks').select('*');

// Insérer une ligne
const { error } = await supabase.from('tasks').insert([{ title: 'Ma tâche' }]);

// Supprimer une ligne
const { error } = await supabase.from('tasks').delete().eq('id', taskId);

// Modifier une ligne
const { error } = await supabase.from('tasks').update({ status: 'done' }).eq('id', taskId);
```

3 — Le routeur (App.jsx)

React Router gère la navigation entre les pages sans rechargement. App.jsx est le chef d'orchestre : il écoute la session Supabase et redirige l'utilisateur selon qu'il est connecté ou non.

```
// src/App.jsx

import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { useEffect, useState } from 'react';
import { supabase } from '../lib/supabase';
import LoginPage from '../pages/LoginPage';
import DashboardPage from '../pages/DashboardPage';

function App() {
  const [session, setSession] = useState(null); // null = pas connecté
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // 1. Vérifier si une session existe déjà (ex : après un refresh)
    supabase.auth.getSession().then(({ data }) => {
      setSession(data.session);
      setLoading(false);
    });

    // 2. Écouter les changements (login, logout)
    const { data: listener } = supabase.auth.onAuthStateChange(
      (_event, session) => setSession(session)
    );

    return () => listener.subscription.unsubscribe();
  }, []);

  if (loading) return <div>Chargement...</div>;

  return (
    <BrowserRouter>
      <Routes>
        { /* Si connecté → /dashboard, sinon → /login */ }
        <Route path='/login'
          element={session ? <Navigate to='/dashboard' /> : <LoginPage />} />
        <Route path='/dashboard'
          element={session ? <DashboardPage session={session} /> : <Navigate
to='/login' />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Route	Comportement
/login	Affiche LoginPage si non connecté. Redirige vers / dashboard si déjà connecté.
/dashboard	Affiche DashboardPage si connecté. Redirige vers /login si non connecté.
/*	Toute URL inconnue → redirection automatique selon l'état de session.

4 — Les pages

4.1 LoginPage.jsx — page publique

Accessible sans être connecté. Gère à la fois la connexion et l'inscription via un bouton bascule.

```
// src/pages/LoginPage.jsx — structure simplifiée
import { useState } from 'react';
import { supabase } from '../lib/supabase';

export default function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [isRegister, setIsRegister] = useState(false); // bascule login/
register
  const [error, setError] = useState('');

  async function handleSubmit(e) {
    e.preventDefault();

    const fn = isRegister
      ? supabase.auth.signUp
      : supabase.auth.signInWithPassword; // même signature !

    const { error } = await fn({ email, password });

    if (error) setError(error.message);

    // Si succès → App.jsx détecte la session et redirige automatiquement
  }

  return ( /* formulaire JSX */ );
}
```

4.2 DashboardPage.jsx — page protégée

Accessible uniquement si connecté (garanti par App.jsx). Reçoit la session en prop et affiche la navigation par onglets.

```
// src/pages/DashboardPage.jsx - structure simplifiée

import { useState, useEffect } from 'react';
import { supabase } from '../lib/supabase';
import TaskList from '../components/TaskList';
import UserTable from '../components/UserTable';

export default function DashboardPage({ session }) {

  const [tab, setTab] = useState('tasks'); // onglet actif
  const [boardId, setBoardId] = useState(null);
  const [users, setUsers] = useState([]);

  useEffect(() => {

    // Charger le premier board de l'utilisateur

    supabase.from('boards').select('id').limit(1)

      .then(({ data }) => data?.[0] && setBoardId(data[0].id));

  }, []);

  async function handleLogout() {

    await supabase.auth.signOut();

    // App.jsx détecte session=null et redirige vers /login

  }

  return (

    <div>

      <header> /* Navbar + bouton déconnexion */ </header>

      <nav> /* Onglets Tâches / Utilisateurs */ </nav>

      {tab === 'tasks' && <TaskList boardId={boardId} />}

      {tab === 'users' && <UserTable users={users} />}

    </div>

  );

}
```


5 — Les composants

5.1 TaskCard.jsx

Reçoit un objet `task` en prop et affiche toutes ses informations. N'écrit jamais en BDD lui-même — il remonte les actions au parent via des callbacks.

```
// src/components/TaskCard.jsx

export default function TaskCard({ task, onDelete }) {
  // task.title, task.status, task.priority, task.due_date
  // task.categories (objet joint via .select('*', categories('*'))
  return (
    <div style={{ border: '2px solid ...', borderRadius: '10px' }}>
      <h3>{task.title}</h3>
      <span>{task.status}</span>
      <span>{task.priority}</span>
      {task.categories && <span>{task.categories.name}</span>}
      <button onClick={() => onDelete(task.id)}>Supprimer</button>
    </div>
  );
}
```

5.2 TaskForm.jsx

Formulaire contrôlé React. À la soumission, envoie un INSERT vers Supabase puis appelle `onCreated()` pour que le parent recharge la liste.

```
// src/components/TaskForm.jsx

export default function TaskForm({ boardId, onCreated }) {

  const [title, setTitle] = useState('');

  // ... autres champs (status, priority, categoryId, dueDate)

  async function handleSubmit(e) {

    e.preventDefault(); // empêche le
    rechargement

    const { data: { user } } = await supabase.auth.getUser();

    const { error } = await supabase.from('tasks').insert([
      {
        title, status, priority, board_id: boardId, created_by: user.id,
        category_id: categoryId || null, due_date: dueDate || null,
      }
    ]);

    if (!error) { resetForm(); onCreated(); } // remonte l'événement
  }

  return ( <form onSubmit={handleSubmit}> ... </form> );
}
```

5.3 TaskList.jsx

Orchestre le chargement des tâches et délègue l'affichage à TaskCard et TaskForm. C'est lui qui appelle Supabase, pas les enfants.

```
// src/components/TaskList.jsx

export default function TaskList({ boardId }) {

  const [tasks, setTasks] = useState([]);

  async function fetchTasks() {

    const { data } = await supabase

      .from('tasks')

      .select('*', categories('*')) // jointure FK automatique

      .eq('board_id', boardId)

      .order('created_at', { ascending: false });

    setTasks(data || []);

  }

  useEffect(() => { fetchTasks(); }, [boardId]); // recharge si boardId
change

  async function handleDelete(taskId) {

    await supabase.from('tasks').delete().eq('id', taskId);

    fetchTasks(); // recharger après suppression

  }

  return (

    <div>

      <TaskForm boardId={boardId} onCreate={fetchTasks} />

      {tasks.map(task => (

        <TaskCard key={task.id} task={task} onDelete={handleDelete} />

      )))}

    </div>

  );

}
```

6 — Les hooks React essentiels

Hook	Usage dans KanbanRT	Exemple
<code>useState</code>	Stocker : tâches, session, onglet actif, champs de formulaire.	<pre>const [tasks, setTasks] = useState([]);</pre>
<code>useEffect</code>	Charger les données au montage d'un composant, écouter la session.	<pre>useEffect(() => { fetchTasks(); }, [boardId]);</pre>
<code>useContext</code>	Partager la session ou l'utilisateur sans prop drilling.	<pre>const { user } = useContext(AuthCtx);</pre>

Règle d'or des hooks

On ne peut appeler `useState` et `useEffect` qu'au niveau supérieur d'un composant fonctionnel — jamais dans une condition, une boucle ou une fonction imbriquée. React impose cet ordre pour maintenir la cohérence entre les rendus.

7 — Conventions à respecter

7.1 Nommage

Ce qui se nomme	Convention
Composants React	PascalCase → TaskCard.jsx, LoginPage.jsx
Fonctions JS	camelCase → fetchTasks(), handleSubmit()
Variables d'état	camelCase → const [isLoading, setIsLoading]
Tables Supabase	snake_case → tasks, task_comments, board_id
Variables d'environnement	SCREAMING_SNAKE_CASE → VITE_SUPABASE_URL
Branches Git	kebab-case → feature/task-creation, fix/login-error
Messages de commit	Anglais, impératif présent → Add task deletion button

7.2 JSX — règles de base

- Un seul élément racine : return (<div>...</div>) ou return (<>...</>)
- class → className dans JSX (class est un mot réservé JavaScript)
- Expressions JavaScript entre accolades : <h3>{task.title}</h3>
- Événements en camelCase : onClick, onChange, onSubmit, onDragStart
- Chaque élément de liste doit avoir un attribut key unique : key={task.id}

7.3 Sécurité minimale obligatoire

- Ne jamais mettre les clés Supabase directement dans le code source
- .env.local toujours dans .gitignore — vérifier avec git status avant chaque push
- Utiliser uniquement la clé anon (jamais la clé service_role côté client)
- Activer le Row Level Security (RLS) sur Supabase pour les tables en production
- Ne jamais utiliser eval() ou innerHTML avec des données saisies par l'utilisateur



Rappel déploiement Vercel

Après chaque git push origin main, Vercel redéploie automatiquement.

Les variables VITE_SUPABASE_URL et VITE_SUPABASE_ANON_KEY doivent être ajoutées dans

Settings → Environment Variables sur vercel.com/dashboard.

Sans elles, l'app déployée ne peut pas se connecter à Supabase.